

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
```

```
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A) ImportError
- B) Module loaded
- 10
- C) Module loaded
- Module loaded
- 10
- D) 10
- 10

Q2. What will happen?

```
def fun(a, b=10):
    return a + b
```

```
print(fun(b=5, a=20))
```

- A) TypeError
- B) 15
- C) SyntaxError
- D) 25

Q3. What is the output?

```
def fun(x):
    x += 10
    return x
```

```
x = 5
y = fun(x)
```

```
print(x, y)
```

- A) 5 5
- B) 5 15
- C) 15 15
- D) 15 5

Q4. What is the output?

```
def fun():
    try:
        return 10
    finally:
        return 20
```

```
print(fun())
```

- A) RuntimeError
- B) 10
- C) 20
- D) None

Q5. What will be the output?

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
        global x
        x = 30
```

```
    inner()
    print(x)
```

```
outer()
print(x)
```

- A) 20
- 30
- B) 30
- 20
- C) 20
- 10



D)30

30

Q6. What is the output?

try:

`x = int("abc")`

except ValueError:

`print("A")`

else:

`print("B")`

finally:

`print("C")`

A)A C

B)B C

C)A

D)B

Q7. What will be printed?

try:

`print(10 / 0)`

except ZeroDivisionError:

`print("A")`

except Exception:

`print("B")`

A)A B

B)A

C)Nothing

D)B

Q8. What is the output?

def outer():

`x = 10`

def inner():

`return x + 5`

`return inner`

f = outer()

print(f())

A)10

B)15

C)None

D)NameError

Q9. What is the purpose of this statement?

if __name__ == "__main__":

A)It prevents all imports

B)It creates the main function

C)It prevents syntax errors

D)It identifies code intended to run when the file is executed directly

Q10. What is the output?

try:

`print(10 / 0)`

except Exception:

`print("A")`

except ZeroDivisionError:

`print("B")`

A)B

B)A

C)SyntaxError

D)A B

Q11. What happens here?

from math import sqrt

print(sqrt(36))

A)36

B)6

C)NameError

D)6.0

Q12. What is the output?

def fun(x):

`return x * 2`

a = fun

print(a(5))

A)5

B)10

C)func(5)

D>Error

Q13. What will be printed?

x = 10

f = lambda: x

x = 20

print(f())

A)20

B)None



C)NameError

D)10

Q14. What will be printed?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, b=20))
```

A)SyntaxError

B)20

C)30

D)TypeError

Q15. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)5

B)25

C)5.0

D)sqrt(25)

Q16. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

A)ZeroDivisionError is not an exception

B)Exception is a superclass of ZeroDivisionError

C)except blocks execute randomly

D)ZeroDivisionError occurs before Exception

Q17. What is the output?

```
def fun():
```

```
    try:
```

```
        print("A")
```

```
        raise ValueError
```

```
    except ValueError:
```

```
        print("B")
```

```
        return 10
```

```
    else:
```

```
        print("C")
```

```
        return 20
```

finally:

```
    print("D")
```

```
print(fun())
```

A)A

B

D

10

B)A

B

C

D

10

C)A

B

10

D

D)A

C

D

20

Q18. What will be the output?

```
def fun(x):
```

```
    try:
```

```
        if x < 0:
```

```
            raise ValueError("negative")
```

```
        return x
```

```
    except ValueError:
```

```
        return 100
```

```
    finally:
```

```
        print("cleanup")
```

```
print(fun(-5))
```

A)negative

cleanup

B)cleanup

-5

C)100

cleanup

D)cleanup

100

Q19. What will be printed?

```
class InvalidAgeError(Exception):
```

```
    pass
```



```
def check_age(age):
    if age < 18:
        raise InvalidAgeError("Age is
too low")
    return "Allowed"
```

```
try:
    print(check_age(15))
except InvalidAgeError as e:
    print(type(e).__name__)
    print(e)
```

A)Exception

Age is too low

B)Nothing, because custom exceptions cannot be caught

C)InvalidAgeError

Age is too low

D)InvalidAgeError

15

Q20. What happens when this code runs?

```
def first():
    second()
```

```
def second():
    third()
```

```
def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
```

A)Handled

B)RuntimeError cannot propagate through functions

C)TypeError

D)NameError

Q21. What will be printed?

```
x = 10
```

```
def outer():
    x = 20
```

```
def inner():
    print(x)
```

```
x = 30
inner()
```

```
outer()
```

A)10

B)20

C)NameError

D)30

Q22. What happens when this code executes?

```
def fun():
    print(variable)
    variable = 50
```

```
variable = 10
```

```
fun()
```

A)10

B)50

C)UnboundLocalError

D)NameError

Q23. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

A)First imports only sqrt

B)First provides the module namespace; second imports sqrt directly

C)There is no difference

D)Both import only sqrt

Q24. What happens when this function is called?

```
def fun(a=10, b):
    return a + b
```

A)SyntaxError

B)It executes correctly

C)TypeError

D)NameError



Q25. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

A)24

B)0

C)9

D)10

Q26. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

```
print(fun())
```

A)10 followed by Finally

B)10 only

C)Finally only

D)Finally followed by 10

Q27. What happens here?

```
def fun(a):  
    return a
```

```
print(fun())
```

A)0

B)None

C)TypeError

D)NameError

Q28. What is the output?

```
try:  
    raise ValueError  
except ValueError:  
    print("A")  
    raise
```

A)A, and execution ends normally

B)B

C)No Output

D)A, followed by propagation of the same exception

Q29. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

```
print(fun(3))
```

A)TypeError

B)RecursionError

C)6

D)3

Q30. What does the following function return?

```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A>Hello None

B>Error

C)None

D>Hello

Q31. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

A)20

B)30

C)-10

D)10

Q32. What will be printed?

```
try:  
    x = int("10")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```



- A)B C
- B)B
- C)A C
- D)A

Q33. What will be printed?

```
def make_counter():
    count = 0
```

```
    def counter():
        nonlocal count
        count += 1
        return count
```

```
    return counter
```

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

A)0

1

0

2

B)1

1

1

1

C)1

2

1

3

D)1

2

3

4

Q34. What happens here?

```
try:
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

A)Wrong Value

B)ValueError

C)e

D)Nothing

Q35. What will be the output?

```
x = 10
```

```
def fun():
    print(x)
    x = 20
```

```
fun()
```

A)UnboundLocalError

B)10

C)NameError

D)20

Q36. What will happen?

```
def outer():
    x = 10
```

```
    def inner():
        nonlocal y
        y = 20
```

```
    inner()
```

```
outer()
```

A)20

B)10

C)SyntaxError

D)NameError

Q37. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
            numbers)
    )
)
```

```
print(result)
```

A)[2, 4, 6, 8, 10]

B)[1, 3, 5]

C)[4, 8]

D)[2, 4]



Q38. Which object keeps track of modules that have already been imported during the current Python process?

- A) `__module__`
- B) `sys.imports`
- C) `sys.path`
- D) `sys.modules`

Q39. What will normally happen?

```
import math
import math
```

- A) `SyntaxError`
- B) The module is normally loaded and executed twice
- C) The module is normally loaded once and reused
- D) `ImportError`

Q40. What will be printed?

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
b = fun()
```

```
print(a)
print(b)
```

- A) `[10]`
- `[10]`
- B) `Error`
- C) `[10]`
- `[10, 10]`
- D) `[10, 10]`
- `[10]`

Q41. if test.py has
`print("A")`

```
if __name__ == "__main__":
    print("B")
```

If test.py is executed directly, what will be printed?

- A) Nothing
- B) B
- C) A followed by B
- D) A

Q42. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A) 15
- B) `Error`
- C) 10
- D) 5

Q43. What happens here?

```
class MyError(Exception):
    pass
```

```
def test():
    raise MyError("Something went wrong")
```

```
try:
    test()
except MyError as e:
    print(type(e).__name__)
```

- A) `MyError`
- B) `TypeError`
- C) `Something went wrong`
- D) `Exception`

Q44. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A) B
- B) A
- C) A and B
- D) Nothing

Q45. What is the output?

```
def fun(x):
    if x == 0:
        return
    return x + fun(x - 1)
```



print(fun(3))

- A)5
- B)6
- C)3
- D)None

Q46. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

fun()

- A)A B
- B)Nothing
- C)B
- D)A

Q47. What is the output?

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal x  
        x = x + 5  
        return x  
  
    inner()  
    return x
```

print(outer())

- A)5
- B)15
- C)UnboundLocalError
- D)10

Q48. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample  
  
print(sample.x)
```

sample.display()

- A)Error
- B)x
- display
- C)10
- Hello
- D)Only Hello

Q49. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

- A)The local file can shadow the intended standard-library module
- B)Python automatically renames the local file
- C)Python will always reject it
- D)random cannot be imported

Q50. What happens?

```
def fun(a, b):  
    return a + b
```

print(fun(10, a=20))

- A)TypeError
- B)30
- C)SyntaxError
- D)20

